Лабораторная работа №1

Тема: Установка и работа с Docker

Docker — приложение, упрощающее процесс управления процессами приложений в контейнерах. Контейнеры позволяют запускать приложения в процессах с изолированными ресурсами. Они похожи на виртуальные машины, но контейнеры более портативны, более экономичны в отношении ресурсов и в большей степени зависят от операционной системы хоста.

Для начала работы запускаем в VirtualBox ранее созданную виртуальную машину с Ubuntu 22.04

Задание №1 Установка Docker

В официальном репозитории Ubuntu может быть доступен не самый последний установочный пакет Docker. Чтобы быть уверенными в получении последней версии, мы будем устанавливать Docker из официального репозитория Docker. Для этого мы добавим новый источник пакетов, импортируем GPG-ключ Docker, чтобы убедиться в подлинности загрузок, а затем проведем установку пакета.

1. Обновляем существующие списки пакетов:

```
sudo apt update
```

2. Устанавливаем необходимые пакеты, которые позволят `apt` использовать пакеты через HTTPS:

```
sudo apt install apt-transport-https ca-certificates curl software-properties-common
```

3. Добавляем в систему ключ GPG для официального репозитория Docker:

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
```

4. Добавляем репозиторий Docker в источники `apt`:

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

5. Обновляем существующий список пакетов еще раз, чтобы добавление было распознано:

```
sudo apt update
```

6. Необходимо проверить установку именно из репозитория Docker:

```
apt-cache policy docker-ce
```

7. Устанавливаем Docker:

```
sudo apt install docker-ce
```

8. Проверяем статус:

```
sudo systemctl status docker
```

Чтобы просмотреть все доступные подкоманды, введите:

```
sudo docker
```

Задание №2 Работа с образами Docker

Контейнеры Docker формируются на основе образов Docker. По умолчанию Docker получает эти образы из Docker Hub - центрального реестра Docker, который управляется компанией Docker, являющейся основным разработчиком проекта Docker. Docker Hub позволяет любому пользователю размещать свои собственные образы Docker, поэтому большинство приложений и дистрибутивов Linux, необходимых вам, можно найти и загрузить именно оттуда.

1. Для проверки доступности к изображениям и их загрузке из Docker Hub, вводим:

```
sudo docker run hello-world
```

2. Поиск изображений, доступные в Docker Hub, осуществляются с помощью подкоманды search команды docker. Например, для поиска образа Ubuntu, выполним следующую команду:

```
sudo docker search ubuntu
```

   Скрипт просканирует Docker Hub и вернет список всех изображений, имя которых соответствует строке поиска. В столбце OFFICIAL [OK] указывает на изображение, созданное и поддерживаемое компанией, стоящей за проектом.
3. Определив изображение, которое планируются использовать, загружаем его с помощью подкоманды pull:

```
sudo docker pull ubuntu
```

4. Загруженные изображения можно посмотреть с помощью команды:

```
sudo docker images
```

Задание №3 Запуск Docker- контейнера

1. Запускаем контейнер, используя последний образ Ubuntu. Комбинация ключей -i и -t дает вам интерактивный доступ к контейнеру через оболочку:

```
sudo docker run -it ubuntu
```

2. Запустить любую команду внутри контейнера. Например, обновляем базу данных пакетов внутри контейнера. Вам не нужно добавлять префикс к какой-либо команде sudo, поскольку вы работаете внутри контейнера как пользователь root :

```
apt update
```

3. Затем устанавливаем в него любое приложение

```
apt install nodejs
```

4. После завершения установки убедитесь, что Node.js установлен:

```
node -v
```

5. Выйти из контейнера можно с помощью команды:

```
exit
```

Задание №4 Управление Docker- контейнерами

1. Для просмотра активных контейнеров используется команда:

```
sudo docker ps
```

2. Для просмотра активных и неактивных контейнеров используется команда docker ps с –a переключателем:

```
sudo docker ps -a
```

3. Чтобы просмотреть последний созданный контейнер, передайте ему переключатель -l:

```
sudo docker ps -l
```

4. Чтобы запустить остановленный контейнер, используйте `docker start`, а затем идентификатор контейнера или имя контейнера.
5. Контейнер запустится, и можем с помощью команды `docker ps` увидеть его статус.
6. Чтобы остановить работающий контейнер, используем `docker stop`, а затем идентификатор или имя контейнера.
7. Если контейнер больше не нужен, удаляем его с помощью команды `docker rm`, используя либо идентификатор контейнера, либо имя.

Задание №5 Внесение изменений в контейнере в образ Docker

1. В задании №3 был установлен Node.js внутри контейнера Ubuntu, что позволило создать контейнер на основе образа. Однако, созданный контейнер отличается от исходного образа. В будущем, возможно, вам захочется использовать этот контейнер Node.js как основу для новых изображений. Чтобы сохранить изменения, выполните следующую команду для создания нового экземпляра образа Docker.

```
sudo docker commit -m "What you did to the image" -a "Author Name"
container_id repository/new_image_name
```

Ключ `-m` предназначен для сообщения о фиксации, которое поможет вам и другим узнать, какие изменения вы внесли, а ключ `-a` используется для указания автора.

`container_id` – идентификатор контейнера

`repository` – имя пользователя

`new_image_name` – новое название образа

Пример:

```
sudo docker commit -m "added Node.js" -a "admin" d9b100f2f636 admin/ubuntu-nodejs
```

2. Когда вы фиксируете изображение, новое изображение сохраняется локально на вашем компьютере. При повторном перечислении образов Docker будет показан новый образ, а также старый, на основе которого он был получен:

```
sudo docker images
```

Задание №6 Отправка образов Docker в репозиторий Docker

1. Чтобы отправить образ в Docker Hub или любой другой реестр Docker, там должна быть учетная запись.
2. Чтобы отправить образ, сначала войдем в Docker Hub.

```
sudo docker login -u docker-registry-username
```

`docker-registry-username` – имя пользователя в реестре

Вам будет предложено пройти аутентификацию с использованием пароля Docker Hub. Если вы указали правильный пароль, аутентификация должна пройти успешно.

3. Если ваше имя пользователя в реестре Docker отличается от локального имени пользователя, которое вы использовали для создания образа, вам придется пометить свой образ именем пользователя в реестре. Для примера, приведенного на последнем шаге, вы должны ввести:

```
sudo docker tag admin/ubuntu-nodejs docker-registry-username/ubuntu-nodejs
```

4. Затем нужно отправить свое собственное изображение, используя:

```
sudo docker push docker-registry-username/docker-image-name
```

5. После отправки образа в реестр он должен появиться на панели управления вашей учетной записи.